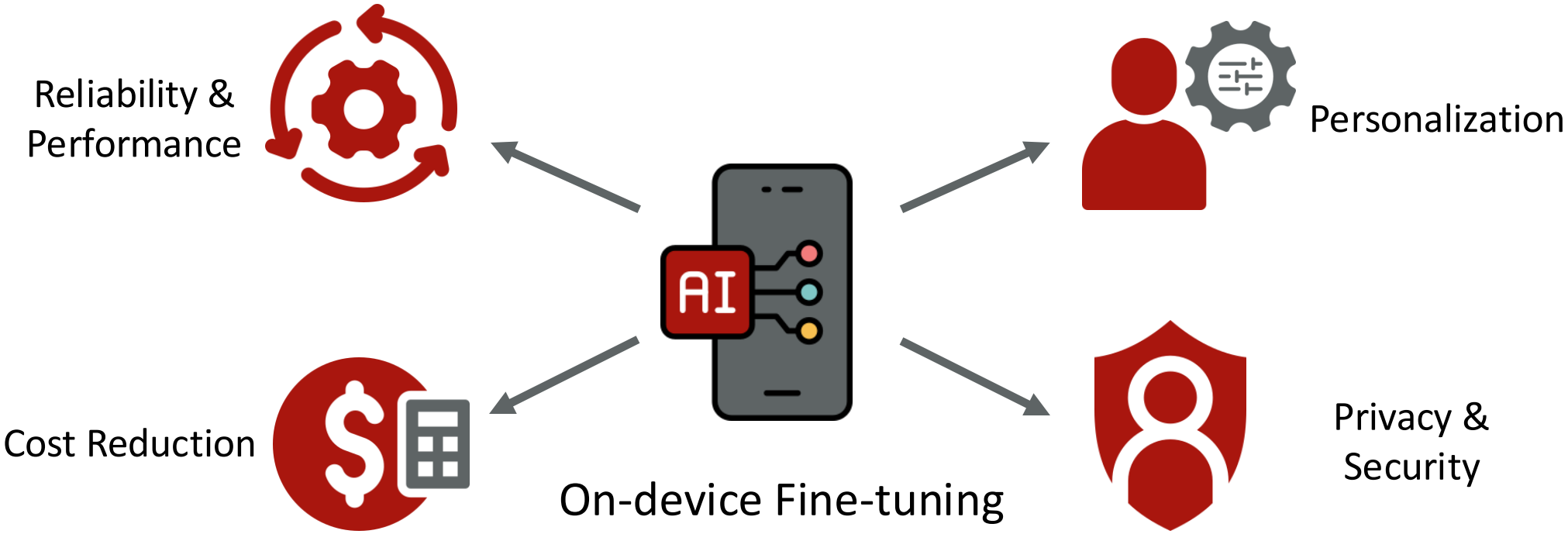
Enabling Efficient On-Device Fine-Tuning of LLMs Using Only Inference Engines

Lei Gao*, Amir Ziashahabi*, Yue Niu, Salman Avestimehr, Murali Annavaram

University of Southern California

On-Device AI


Reliability &
Performance
Personalization
AI
Cost Reduction
Privacy &
Security
On-device Fine-tuning

On-Device Training Challenges

- Memory constraints as a game-stopper

~10GB RAM

Low precision

	Params	Gradients	Optim States	Activations	Total Memory
AdamW	12.55	12.55	75.31	45.61	147.02
SGD	12.55	12.55	25.10	45.61	96.81

Memory usage (GB) when training LLaMA-7B under different settings.
The sequence length and batch size are set to 512 and 8, respectively.

Full Parameter Fine-tuning for Large Language Models with Limited Resources (Lv et al., ACL 2024)

On-Device Training Challenges

- Memory constraints as a game-stopper

~10GB RAM

PEFT (e.g., LoRA)

	Params	Gradients	Optim States	Activations	Total Memory
AdamW	12.55	12.55	75.31	45.61	147.02
SGD	12.55	12.55	25.10	45.61	96.81

Memory usage (GB) when training LLaMA-7B under different settings.
The sequence length and batch size are set to 512 and 8, respectively.

Full Parameter Fine-tuning for Large Language Models with Limited Resources (Lv et al., ACL 2024)

On-Device Training Challenges

- Memory constraints as a game-stopper

Gradient accumulation
Activation checkpointing

~10GB RAM

	Params	Gradients	Optim States	Activations	Total Memory
AdamW	12.55	12.55	75.31	45.61	147.02
SGD	12.55	12.55	25.10	45.61	96.81

Memory usage (GB) when training LLaMA-7B under different settings.
The sequence length and batch size are set to 512 and 8, respectively.

Full Parameter Fine-tuning for Large Language Models with Limited Resources (Lv et al., ACL 2024)

On-Device Training Challenges

- Memory constraints as a game-stopper

~10GB RAM

Sparse update

	Params	Gradients	Optim States	Activations	Total Memory
AdamW	12.55	12.55	75.31	45.61	147.02
SGD	12.55	12.55	25.10	45.61	96.81

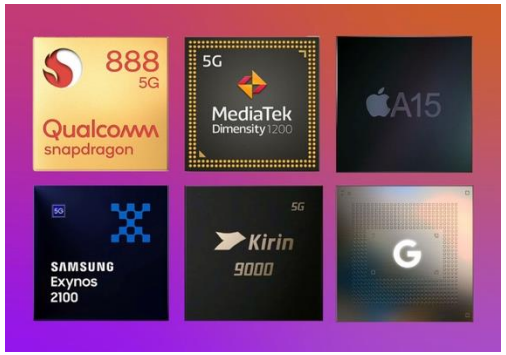
Memory usage (GB) when training LLaMA-7B under different settings.
The sequence length and batch size are set to 512 and 8, respectively.

Full Parameter Fine-tuning for Large Language Models with Limited Resources (Lv et al., ACL 2024)

On-Device Training Challenges

- Lack of on-device training frameworks

CUDA C/C++



On-Device Training Challenges

- Lack of on-device training frameworks

ExecuTorch

MLC

LLAMA C++

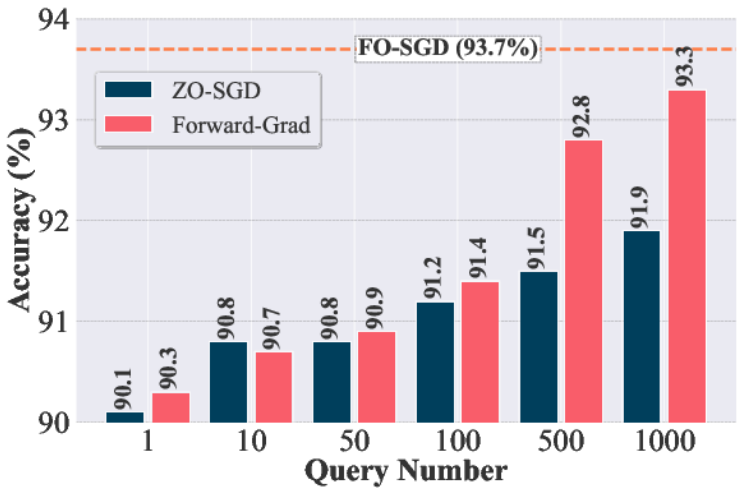
But there are many well-optimized on-device inference frameworks

Zeroth-Order Optimization as a Potential Solution

- Randomized Gradient Estimator (RGE)

$$\hat{\nabla}\mathcal{L}(\theta;\mathcal{B}) = \frac{1}{q}\sum_{i=1}^{q}\left[\frac{\mathcal{L}(\theta+\epsilon z_i;\mathcal{B}) - \mathcal{L}(\theta-\epsilon z_i;\mathcal{B})}{2\epsilon} z_i\right]$$

- No need of backpropagation
- Same memory cost as inference
- Improved accuracy with larger query number



LoRA-based fine-tuning accuracy of OPT-1.3B on SST2

Revisiting Zeroth-Order Optimization for Memory-Efficient LLM Fine-Tuning: A Benchmark (Zhang et al., ICML 2024)

Zeroth-Order Optimization as a Potential Solution

- Randomized Gradient Estimator (RGE)

$$\hat{\nabla}\mathcal{L}(\theta;\mathcal{B}) = \frac{1}{q}\sum_{i=1}^{q}\left[\frac{\mathcal{L}(\theta+\epsilon z_i;\mathcal{B}) - \mathcal{L}(\theta-\epsilon z_i;\mathcal{B})}{2\epsilon} z_i\right]$$

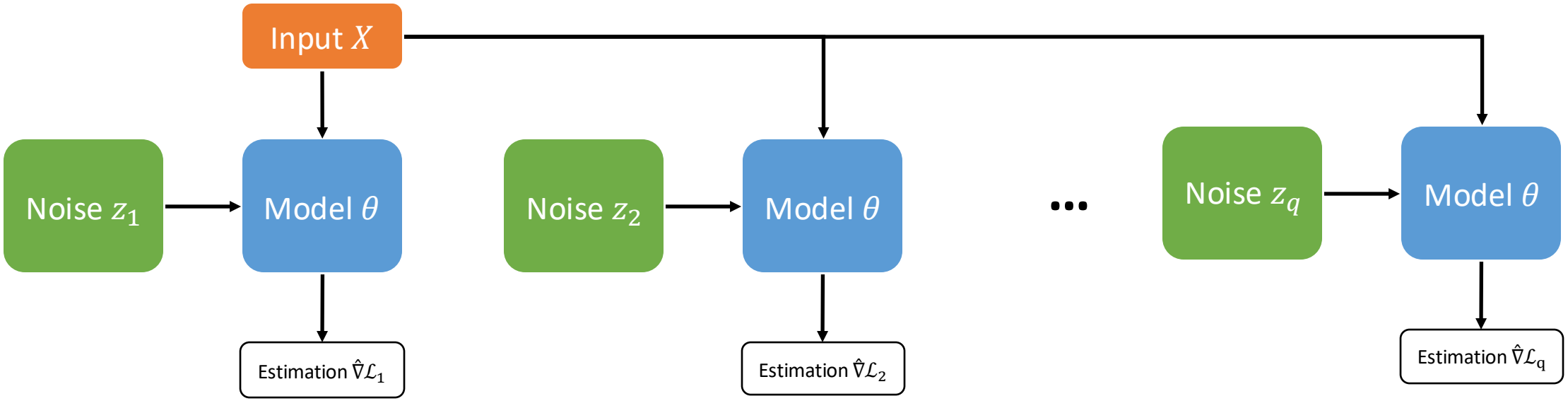
- However, each estimation takes 2q forward passes
- Considerable increase in runtime when executed sequentially

Need a parallelized solution

Revisiting Zeroth-Order Optimization for Memory-Efficient LLM Fine-Tuning: A Benchmark (Zhang et al., ICML 2024)

Naïve solution

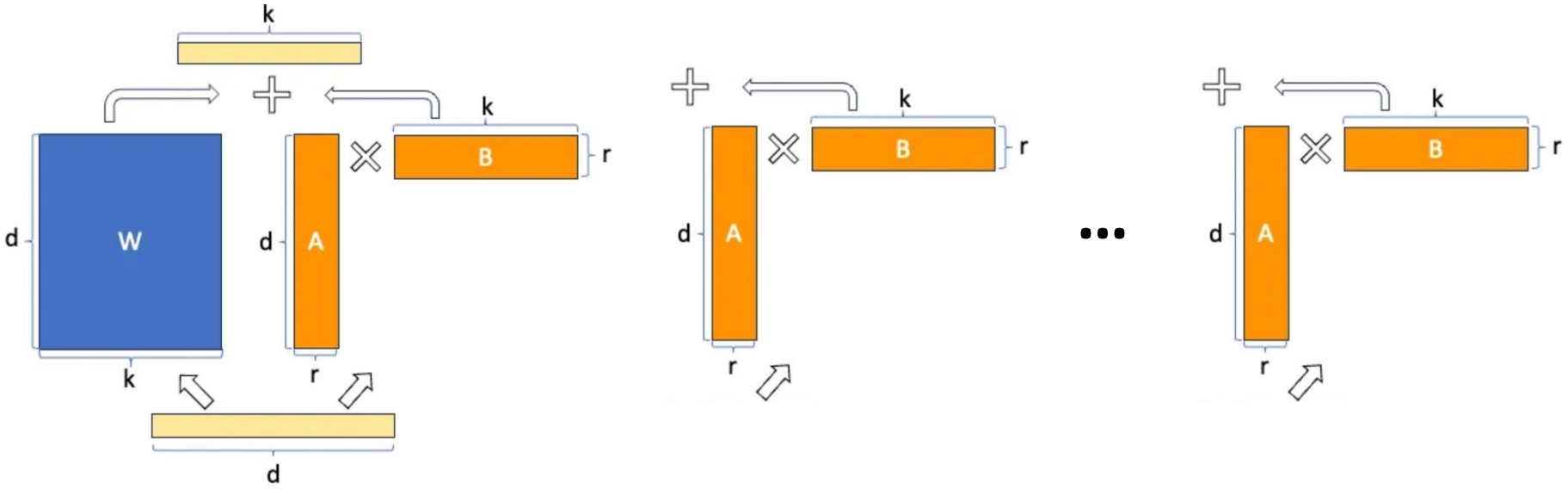
- Duplicate the model and run q separate tasks in parallel for each training step



Problem 1: memory overhead if duplicating the full parameter space
Problem 2: computation overhead if forwarding all models at the same time

Overcome Memory Overhead

- Adapt parameter-efficient fine-tuning and only duplicate trainable parameters

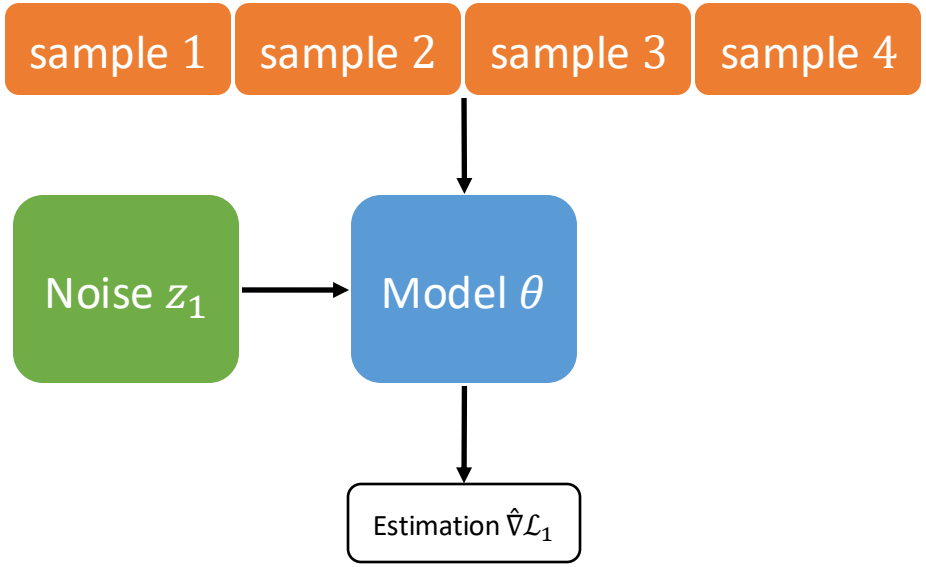


Low-rank adaptation (LoRA)

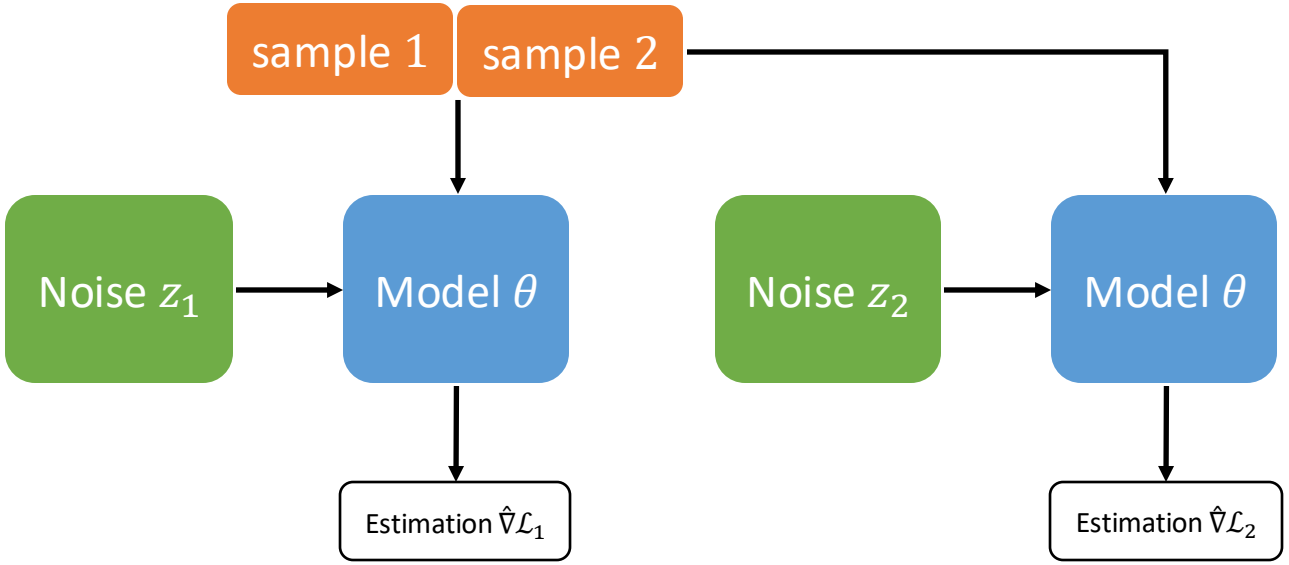
LoRA: Low-Rank Adaptation of Large Language Models (Hu et al., ICLR 2022)

Overcome Computation Overhead

- Reduce batch size proportionally as increasing the query number



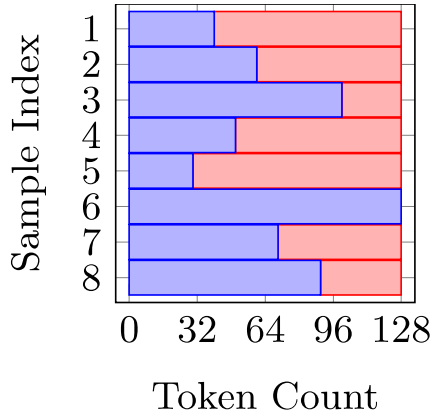
$q = 1, B = 4$



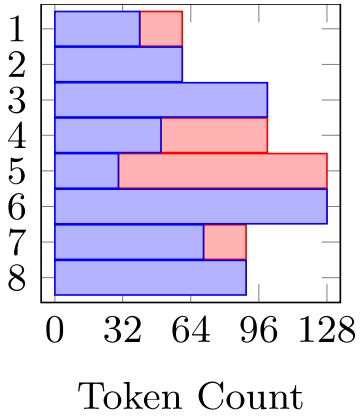
$q = 2, B = 2$

Overcome Computation Overhead

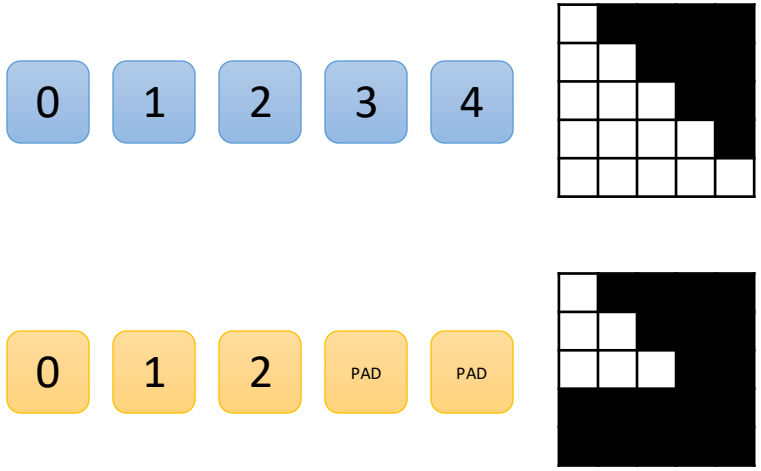
- Reduced number of padding tokens as reducing batch size



(a) Padding on batch size of 8.



(b) Padding on batch size of 2.



Prompt tokens

Causal masking

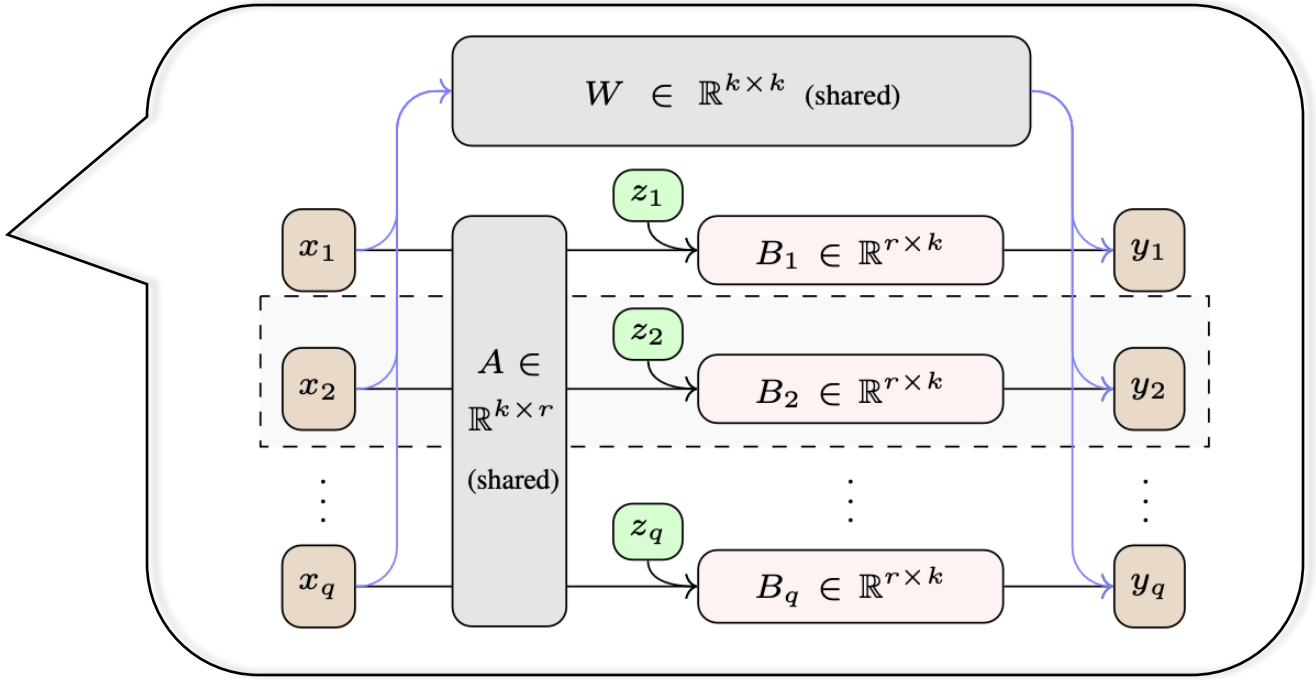
Less amount of wasteful computation

Parallelized Randomized Gradient Estimator (P-RGE)

(P-RGE) Algorithm.

Input: learnable parameters $\boldsymbol{\theta}_l \in \mathbb{R}^{d_l}$, frozen parameters $\boldsymbol{\theta}_f \in \mathbb{R}^{d_f}$, loss $\mathcal{L} : \mathbb{R}^{d_l} \times \mathbb{R}^{d_f} \to \mathbb{R}$, step budget T, function query budget q, perturbation scale ϵ, learning rate η
for $t = 1$ to T **do**
  Sample batch $\boldsymbol{\mathcal{B}} \subset \mathcal{D}$
  for $i = 1$ to q **do in parallel** ▷ Outer Loop
    Sample random seed s_i
    Generate random noise $\boldsymbol{z}_i \sim \mathcal{N}(0, \boldsymbol{I}_{d_l})$ using s_i
    for $k \in \{+1, -1\}$ **do in parallel** ▷ Inner Loop
      Perturb parameters: $\boldsymbol{\theta}_l^{(k)} = \boldsymbol{\theta}_l + k\epsilon\boldsymbol{z}_i$
      Compute loss: $\ell^{(k)} = \mathcal{L}((\boldsymbol{\theta}_l^{(k)}, \boldsymbol{\theta}_f); \boldsymbol{\mathcal{B}})$
    end for
    Get projected gradient: $g_i = \dfrac{\ell^{(+1)} - \ell^{(-1)}}{2\epsilon}$
    Store s_i and g_i
  end for
  Update parameters: $\boldsymbol{\theta}_l \leftarrow \boldsymbol{\theta}_l - \eta \left(\frac{1}{q} \sum_{i=1}^{q} g_i \boldsymbol{z}_i \right)$
end for

Outer-loop parallelization (LoRA-FA based)



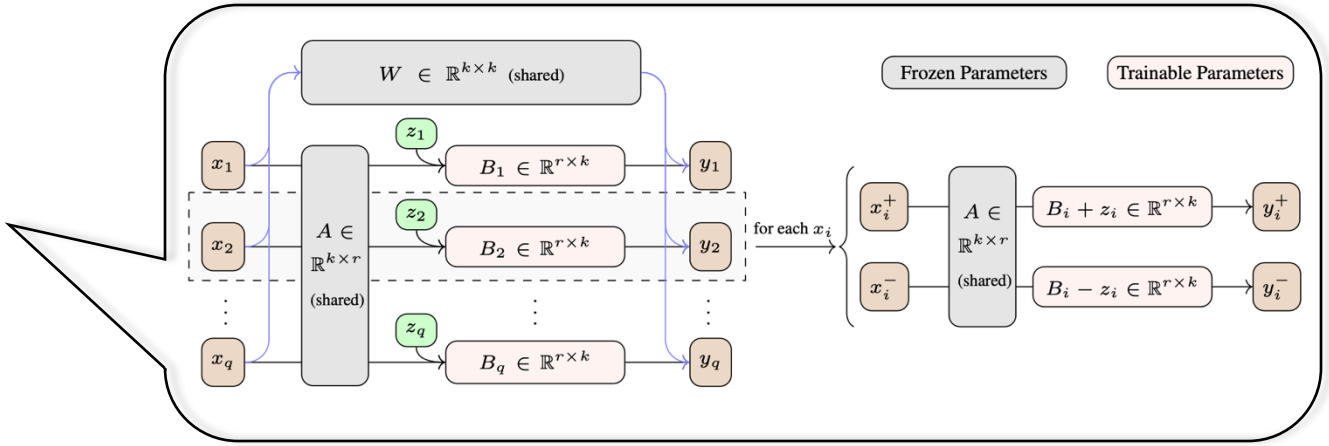
Weights reused across different queries → reduced memory I/O

Parallelized Randomized Gradient Estimator (P-RGE)

(P-RGE) Algorithm.

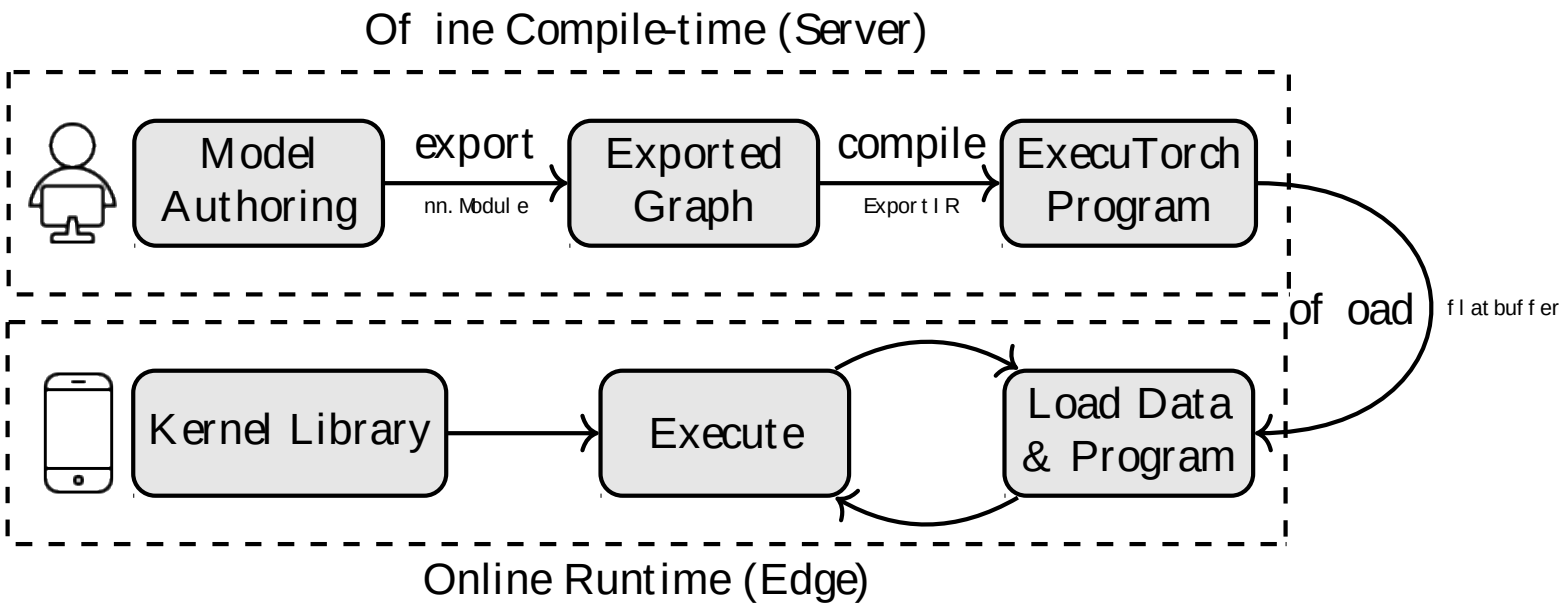
Input: learnable parameters $\boldsymbol{\theta}_l \in \mathbb{R}^{d_l}$, frozen parameters $\boldsymbol{\theta}_f \in \mathbb{R}^{d_f}$, loss $\mathcal{L} : \mathbb{R}^{d_l} \times \mathbb{R}^{d_f} \to \mathbb{R}$, step budget T, function query budget q, perturbation scale ϵ, learning rate η
for $t = 1$ to T **do**
  Sample batch $\boldsymbol{\mathcal{B}} \subset \mathcal{D}$
  for $i = 1$ to q **do in parallel** ▷ Outer Loop
    Sample random seed s_i
    Generate random noise $\boldsymbol{z}_i \sim \mathcal{N}(0, \boldsymbol{I}_{d_l})$ using s_i
    for $k \in \{+1, -1\}$ **do in parallel** ▷ Inner Loop
      Perturb parameters: $\boldsymbol{\theta}_l^{(k)} = \boldsymbol{\theta}_l + k\epsilon\boldsymbol{z}_i$
      Compute loss: $\ell^{(k)} = \mathcal{L}((\boldsymbol{\theta}_l^{(k)}, \boldsymbol{\theta}_f); \boldsymbol{\mathcal{B}})$
    end for
    Get projected gradient: $g_i = \dfrac{\ell^{(+1)} - \ell^{(-1)}}{2\epsilon}$
    Store s_i and g_i
  end for
  Update parameters: $\boldsymbol{\theta}_l \leftarrow \boldsymbol{\theta}_l - \eta \left(\frac{1}{q} \sum_{i=1}^{q} g_i \boldsymbol{z}_i \right)$
end for

Inner-loop parallelization (LoRA-FA based)



Weights reused across two forward passes → further reduced memory I/O

Leverage On-Device Inference Framework



Model inference on-device deployment via ExecuTorch

Leverage On-Device Inference Framework

(P-RGE) Algorithm.

Input: learnable parameters $\boldsymbol{\theta}_l \in \mathbb{R}^{d_l}$, frozen parameters $\boldsymbol{\theta}_f \in \mathbb{R}^{d_f}$, loss $\mathcal{L} : \mathbb{R}^{d_l} \times \mathbb{R}^{d_f} \to \mathbb{R}$, step budget T, function query budget q, perturbation scale ϵ, learning rate η
for $t = 1$ to T **do**
  Sample batch $\boldsymbol{\mathcal{B}} \subset \mathcal{D}$
  for $i = 1$ to q **do in parallel** ▷ Outer Loop
    Sample random seed s_i
    Generate random noise $\boldsymbol{z}_i \sim \mathcal{N}(0, \boldsymbol{I}_{d_l})$ using s_i
    for $k \in \{+1, -1\}$ **do in parallel** ▷ Inner Loop
      Perturb parameters: $\boldsymbol{\theta}_l^{(k)} = \boldsymbol{\theta}_l + k\epsilon\boldsymbol{z}_i$
      Compute loss: $\ell^{(k)} = \mathcal{L}((\boldsymbol{\theta}_l^{(k)}, \boldsymbol{\theta}_f); \boldsymbol{\mathcal{B}})$
    end for
    Get projected gradient: $g_i = \dfrac{\ell^{(+1)} - \ell^{(-1)}}{2\epsilon}$
    Store s_i and g_i
  end for
  Update parameters: $\boldsymbol{\theta}_l \leftarrow \boldsymbol{\theta}_l - \eta \left(\frac{1}{q} \sum_{i=1}^{q} g_i \boldsymbol{z}_i \right)$
end for

Memory-efficient ZO (random seed trick)

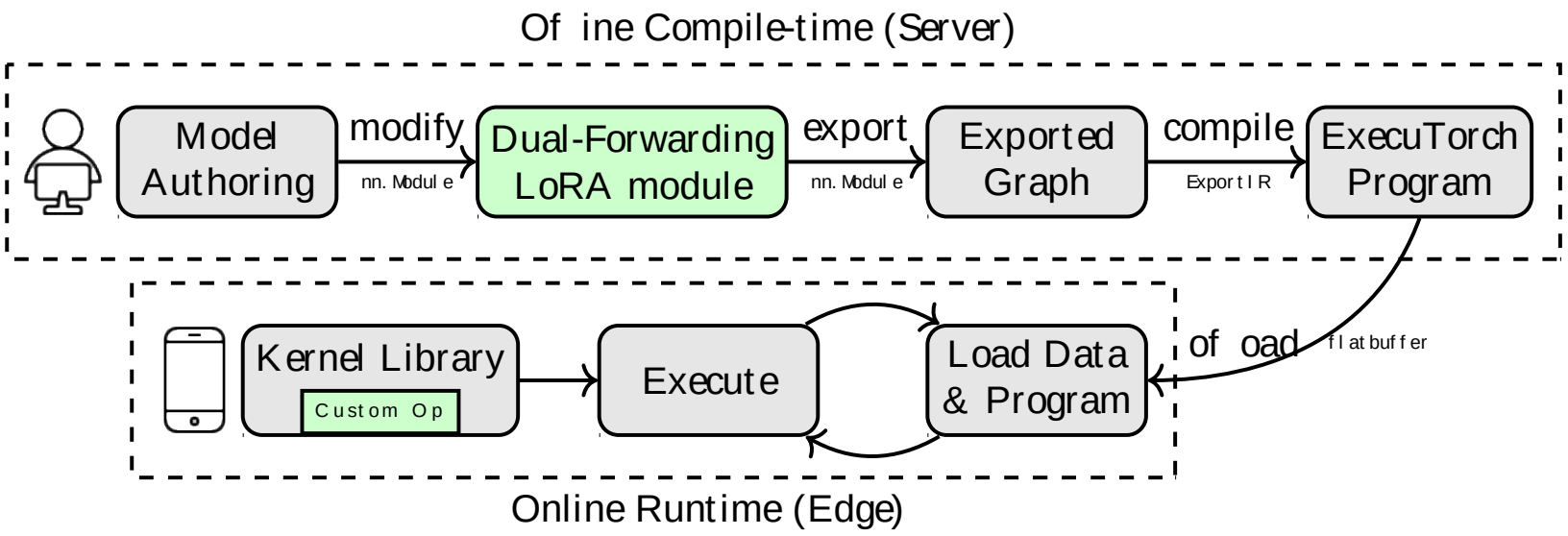
- Memory cost of random noise: $O(d) \to O(1)$
- **Not an exportable Tensor operation**

Leverage On-Device Inference Framework

- Encode P-RGE into a LoRA module `forward` function

Algorithm 2 Dual-forwarding LoRA-FA Module Definition

1: **Input:** $\mathbf{x} \in \mathbb{R}^{2\times \text{seq_len}\times k}$, $\mathbf{A} \in \mathbb{R}^{k\times r}$, $\mathbf{B} \in \mathbb{R}^{2\times r\times k}$, $\mathbf{W}^{k\times k}$, learning rate η, perturbation scale ϵ, projected gradient g
2: $\text{diff} = \frac{\mathbf{B}[0]-\mathbf{B}[1]}{2}$
3: $\text{update} = \eta \cdot g \cdot \frac{\text{diff}}{\epsilon}$
4: $\mathbf{z} = \epsilon \cdot randn_like(\mathbf{B}[0])$
5: $\mathbf{B}[0] = \mathbf{B}[0] - \text{diff} - \text{update} + \mathbf{z}$
6: $\mathbf{B}[1] = \mathbf{B}[1] + \text{diff} - \text{update} - \mathbf{z}$
7: $\text{output} = \mathbf{xW} + bmm(\mathbf{xA}, \mathbf{B})$
8: **Return:** output



Achieve fine-tuning via ExecuTorch with minimal modifications

Experiments – server-side model performance

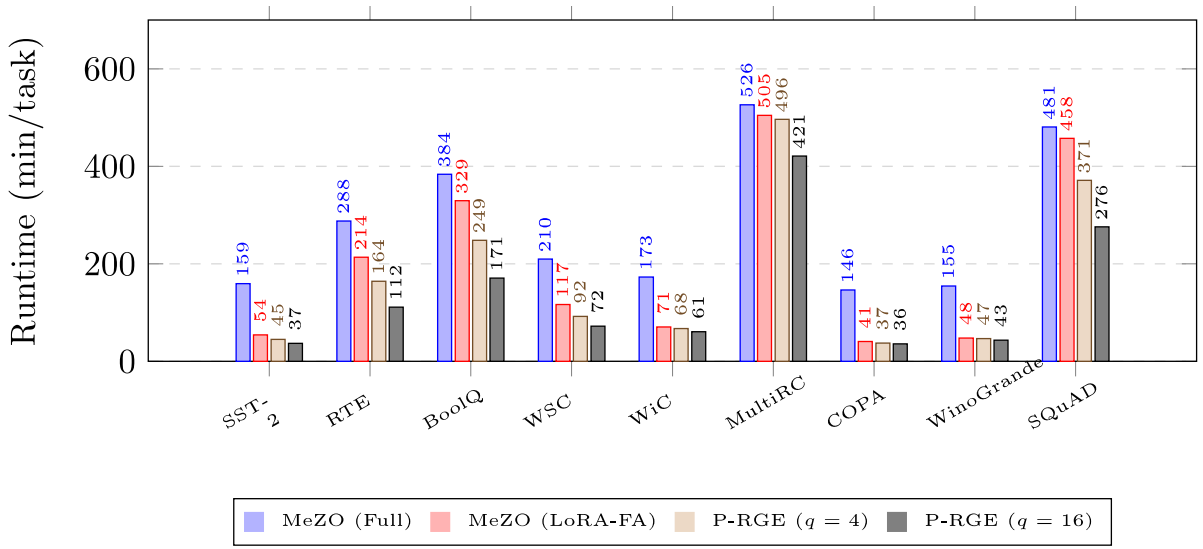
- Performance of fine-tuning Llama2-7B on different tasks with different optimizers

Methods		SST-2	RTE	BoolQ	WSC	WiC	MultiRC	COPA	WinoGrande	SQuAD
Zero-shot		58.0	59.2	71.9	51.9	50.0	54.6	79.0	62.7	23.8
FO-Adam	Full	92.5	78.7	80.6	63.4	67.2	71.7	81.0	68.2	79.2
	LoRA-FA	96.0	88.1	85.7	79.8	75.1	84.2	87.0	71.8	77.2
ZO-SGD	MeZO (Full)	93.7	72.6	81.1	64.4	54.6	69.9	81.0	64.4	76.9
	MeZO (LoRA-FA)	94.4	72.9	80.7	64.4	60.0	73.1	86.0	64.5	79.5
	P-RGE ($q = 4$)	**94.9**	76.2	**83.0**	63.5	**64.4**	**74.5**	85.0	65.4	**79.8**
	P-RGE ($q = 16$)	94.2	**77.6**	82.5	**65.4**	62.9	74.9	**87.0**	**65.7**	78.6

P-RGE outperforms the baseline MeZO in all tasks, showing the effectiveness of multi-query RGE

Experiments – server-side system performance

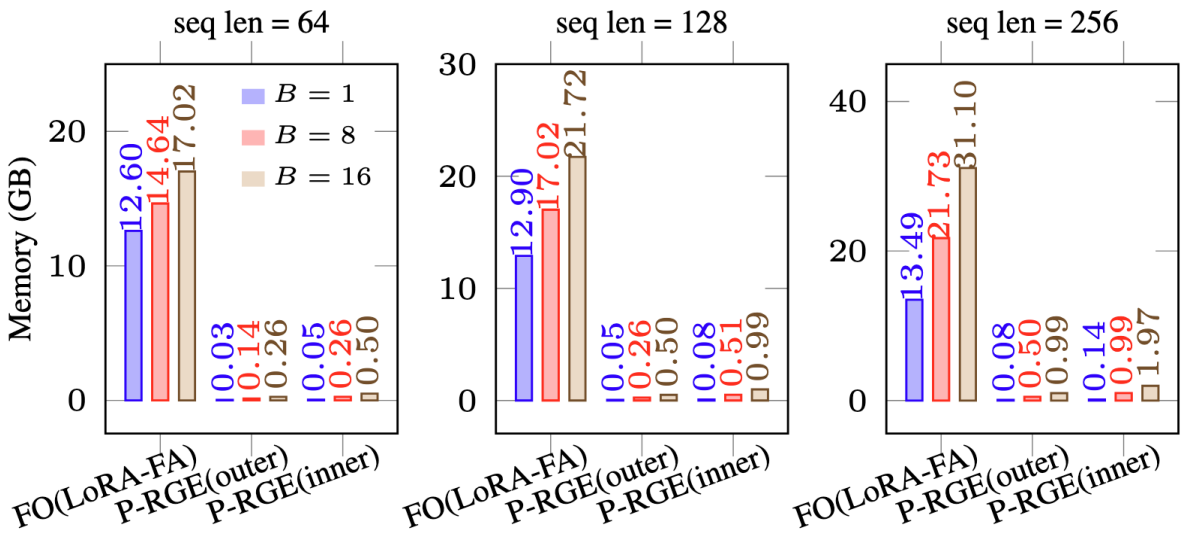
- End-to-end wall-clock time of fine-tuning Llama2-7B for various configurations across tasks



P-RGE achieves faster end-to-end wall-clock time compared to MeZO (q=1)

Experiments – server-side system performance

- Peak memory usage of Llama2-7B for different sequence lengths and batch sizes



Memory usage (GB) for storing model weights

Quantization	FP16	INT8	NF4
Llama2-7b	12.56	6.52	3.50

P-RGE occurs with minimal memory overhead

Experiments – edge-side system performance

- Runtime (sec/step) and speedup ratio of inner-loop parallelization on Jetson GPU backend for Llama2-7B with NF4 quantization

Model	Sequence length	64				128			
	Batch size	1	2	4	8	1	2	4	8
Llama2-7B	P-RGE (outer)	3.10	3.37	4.44	6.46	3.37	4.44	6.47	10.83
	P-RGE (inner)	1.69	2.22	3.22	5.38	2.22	3.22	5.37	8.60
	Speedup ratio	1.83	1.52	1.38	1.20	1.52	1.38	1.21	1.26

- Runtime and memory usage of dual-forwarding implementation on Android NPU backend for TinyLlama-1.1B

Sequence length	64				128			
Effective batch size	2	4	8	16	2	4	8	16
Runtime (sec/step)	1.04	2.34	4.70	10.43	2.49	4.83	10.36	15.73
Memory (GB)	3.36	3.53	3.75	3.88	3.43	3.68	3.91	4.46

Future Work

- Adapt P-RGE to other zero-order optimization techniques
- Incorporate low-precision methods for intermediate activations
- Explore adaptive scheduling of ZO optimization to accommodate dynamic resource availability on the edge